

Gmail API Order Notification System

Real Implementation Guide — OAuth2 + Laravel Queues

Project: Parfum
Store

Stack: Laravel 11 + Gmail
REST API

Date: March 31,
2026

Status: Production
Ready

Gmail API Order Notification System — Real Implementation Guide



Table of Contents

1. [Step 0 — How to Obtain Gmail API Credentials](#)
2. [Helper Scripts — Token Acquisition Files](#)
3. [google-credentials.json — Role & Structure](#)
4. [Overview](#)
5. [Real Architecture](#)
6. [Real File Structure](#)
7. [File 1 — GmailService.php](#)
8. [File 2 — SendAdminOrderEmail.php](#)
9. [File 3 — InvoiceService.php](#)
10. [File 4 — OrderService.php \(dispatch\)](#)
11. [File 5 — invoices/order.blade.php](#)
12. [Environment Configuration \(.env\)](#)
13. [Operation Flow Step-by-Step](#)
14. [Queue Worker](#)
15. [Troubleshooting](#)

Step 0 — How to Obtain Gmail API Credentials (One-Time Setup)

This system uses **OAuth2 via the Gmail REST API**. There are two parts to configure once:

PART A — Google Cloud Console (Gmail API credentials)

This gives you `GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`, and `GOOGLE_REFRESH_TOKEN`.

Step 1 — Create a Google Cloud Project

1. Go to → <https://console.cloud.google.com>
2. Click **"Select a project"** → **"New Project"**
3. Name it: `Parfum Store` → Click **Create**

Step 2 — Enable Gmail API

1. In the left menu go to → **APIs & Services** → **Library**
2. Search: `Gmail API`
3. Click on it → Click **Enable**

Step 3 — Create OAuth2 Credentials

1. Go to → **APIs & Services** → **Credentials**
2. Click **" + Create Credentials "** → Choose **"OAuth client ID"**
3. If prompted, configure the **OAuth consent screen** first:
 - User Type: **External**
 - App name: `Parfum Store`
 - Support email: `zakariaaalbad200@gmail.com`
 - Scopes: add `https://www.googleapis.com/auth/gmail.send`
 - Test users: add `zakariaaalbad200@gmail.com`
 - Click **Save and Continue**
4. Back in Credentials → Application type: **Desktop app**
5. Name: `Parfum Store Backend`
6. Click **Create**
7. Click **Download JSON** → save as `google-credentials.json`

This JSON contains:

```
{
  "installed": {
    "client_id": "XXXXXXXXX.apps.googleusercontent.com",
    "client_secret": "GOCSPX-XXXXXXXXX",
    ...
  }
}
```

Copy `client_id` → `GOOGLE_CLIENT_ID` in `.env` Copy `client_secret` → `GOOGLE_CLIENT_SECRET` in `.env`

Step 4 — Get the Refresh Token (One-Time OAuth Flow)

1. Open this URL in your browser (replace CLIENT_ID):

```
https://accounts.google.com/o/oauth2/auth
?client_id=YOUR_CLIENT_ID
&redirect_uri=http://localhost:9090
&response_type=code
&scope=https://www.googleapis.com/auth/gmail.send
&access_type=offline
&prompt=consent
```

2. Sign in with `zakariaaalbad200@gmail.com` → Click **Allow**
3. Browser redirects to `http://localhost:9090/?code=4/0Adeu5BV...`
4. Run the artisan command to auto-capture and exchange code:

```
php artisan gmail:get-token
```

5. The command automatically:
 - Starts a PHP server on port 9090
 - Catches the redirect with the `code`
 - Exchanges it via `POST https://oauth2.googleapis.com/token`
 - Writes `GOOGLE_REFRESH_TOKEN=...` into your `.env` automatically

✅ **The refresh token never expires** unless you revoke it in Google Console. You only need to do this **once per deployment**.

PART B — Gmail App Password (Optional / SMTP fallback only)

⚠ The active system uses **Gmail REST API**, NOT SMTP. The App Password (`MAIL_PASSWORD`) is only kept as a fallback configuration.

If you ever need it:

Step 1 — Enable 2-Step Verification

1. Go to → <https://myaccount.google.com/security>
2. Under "How you sign in to Google" → Click **"2-Step Verification"** → Enable it

Step 2 — Generate App Password

1. Go to → <https://myaccount.google.com/apppasswords>
2. In the **"App name"** field type: `Parfum Store`
3. Click **Create**
4. Google displays a **16-character password** (e.g. `iott drjt fcta xbqr`)
5. Copy it (without spaces) → paste into `.env` :

```
MAIL_PASSWORD=iottdrjtfctaxbqr
```

⚠ App Passwords may be blocked by Google if your account is flagged for security. The Gmail REST API with OAuth2 (PART A) is the **recommended and production-grade** approach.

🔧 Helper Scripts — Token Acquisition Files

Three standalone PHP files exist in the `backend/` root to help obtain the OAuth2 refresh token. Each serves a specific stage of the OAuth flow. They are **one-time setup tools** — not part of the production runtime.

📄 `get-google-token.php`

Path: `backend/get-google-token.php` **Purpose:** Interactive CLI script. Run from terminal once to obtain the refresh token manually. **Method:** User pastes the redirect URL into the terminal after browser authentication. **Dependency:** Uses Guzzle (already installed via `composer.json`).

How it works — step by step:

```
Step 1: Script builds the Google OAuth2 authorization URL using:
- client_id (hardcoded from google-credentials.json)
- redirect_uri = http://localhost
- scope = https://www.googleapis.com/auth/gmail.send
- access_type = offline ← REQUIRED to receive refresh_token
- prompt = consent      ← Forces Google to always return refresh_token

Step 2: Script prints the URL and waits for user input (fgets(STDIN))

Step 3: User opens URL in browser → signs in → browser redirects to:
http://localhost/?code=4/0Adeu5BV... (page won't load – normal)

Step 4: User copies full URL from browser address bar
and pastes it into the terminal

Step 5: Script extracts the `code` parameter from the pasted URL using:
parse_str(parse_url($redirectUrl, PHP_URL_QUERY), $parsedQuery)

Step 6: Script sends POST to https://oauth2.googleapis.com/token via Guzzle:
- code          = the extracted code
- client_id     = from credentials
- client_secret = from credentials
- redirect_uri  = http://localhost
- grant_type    = authorization_code

Step 7: Google responds with JSON containing:
{ "refresh_token": "1//03rLluk...", "access_token": "ya29...", ... }

Step 8: Script writes GOOGLE_REFRESH_TOKEN=... directly into .env
using file_get_contents + preg_replace + file_put_contents
```

Run it:

```
cd backend
php get-google-token.php
```

⚠ **Limitation:** Requires user to manually paste the redirect URL. Fails if `http://localhost` is occupied by another app (e.g. your frontend on port 3000). In that case, use `exchange_token_server.php` instead.

 `exchange_token_server.php`

Path: `backend/exchange_token_server.php` **Purpose:** Auto-capture server. Starts a tiny PHP web server on port 9090 that automatically catches the OAuth redirect and exchanges the code — no manual URL

copy needed. **Method:** PHP's built-in server listens on `http://localhost:9090`, Google redirects directly to it. **Dependency:** Uses PHP's native `curl` functions.

How it works — step by step:

```
Step 1: An artisan command (gmail:get-token) starts PHP's built-in server:
        php -S localhost:9090 exchange_token_server.php

Step 2: The OAuth URL is built with redirect_uri = http://localhost:9090
        User opens URL in browser → signs in → browser redirects to:
        http://localhost:9090/?code=4/0Adeu5BV...

Step 3: exchange_token_server.php loads automatically (it IS the server).
        It checks: if (!isset($_GET['code'])) → returns 400 error page

Step 4: If code is present, sends POST to https://oauth2.googleapis.com/token
        using curl with params:
        - code, client_id, client_secret, redirect_uri, grant_type

Step 5: On success, parses the JSON response for `refresh_token`

Step 6: Writes the token to .env:
        - If GOOGLE_REFRESH_TOKEN= exists → replaces it using preg_replace
        - If not found → appends it at end of file

Step 7: Runs php artisan config:clear via shell_exec to clear config cache

Step 8: Returns a green HTML success page in the browser:
        "✅ Success! Refresh token saved to .env"

Step 9: On failure → returns red HTML error page with raw API response
```

Run it (via artisan command):

```
php artisan gmail:get-token
```

The artisan command automatically starts the server and prints the OAuth URL.

✅ **Best option** — fully automatic, no manual copy needed. The browser redirects to port 9090 which nothing else uses.

 `exchange_token.php`

Path: `backend/exchange_token.php` **Purpose:** Emergency/debug one-shot script. Hardcodes a specific auth code and immediately exchanges it for a refresh token. **Method:** Direct curl POST to Google's

token endpoint with a fixed code. **Dependency:** PHP native `curl`.

How it works — step by step:

Step 1: Hardcoded OAuth code in the script (obtained manually from redirect URL):
`'code' => '4/0Aci98E9dfxvwJNiDvx4dCVKkWB17YRuobpahM0nh...'`

Step 2: Sends POST via `curl` to `https://oauth2.googleapis.com/token` with:

- `code` = hardcoded auth code
- `client_id` = from `google-credentials.json`
- `client_secret` = from `google-credentials.json`
- `redirect_uri` = `http://localhost:9090`
- `grant_type` = `authorization_code`

Step 3: Parses JSON response:

- If `refresh_token` found → prints it + writes to `.env`
- If error → prints HTTP code + full raw response for debugging

Step 4: Auto-writes `GOOGLE_REFRESH_TOKEN` to `.env` using:
`preg_replace` or `append`

Run it:

```
cd backend
php exchange_token.php
```

⚠ Important: Auth codes expire in ~60 seconds. This script only works if run **immediately** after copying the auth code from the browser URL. Used once during debugging when `exchange_token_server.php` was not yet available.

🔍 Comparison of All Three Scripts

Script	Method	Requires User Input	Best Use
<code>get-google-token.php</code>	CLI + Guzzle	Yes — paste URL	When port 9090 is free but <code>localhost:3000</code> is busy
<code>exchange_token_server.php</code>	PHP server + <code>curl</code>	No — fully auto	Recommended — production setup
<code>exchange_token.php</code>	Direct <code>curl</code> + <code>hardcode</code>	Hardcoded code	Emergency debug only

google-credentials.json

Path: backend/storage/app/google-credentials.json **Purpose:** Stores the OAuth2 client credentials downloaded from Google Cloud Console. **Security:** Never commit this file to git. Contains your client secret.

Structure:

```
{
  "installed": {
    "client_id": "379053566763-8ikbpbcmohpls89fddv859n99219ghq.apps.googleusercontent.com",
    "project_id": "focus-reality-486412-a2",
    "auth_uri": "https://accounts.google.com/o/oauth2/auth",
    "token_uri": "https://oauth2.googleapis.com/token",
    "auth_provider_x509_cert_url": "https://www.googleapis.com/oauth2/v1/certs",
    "client_secret": "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",
    "redirect_uris": ["http://localhost"]
  }
}
```

Each field explained:

Field	Role
client_id	Unique ID of your Google Cloud OAuth2 app → GOOGLE_CLIENT_ID in .env
project_id	Your Google Cloud project name (informational only)
auth_uri	Google's authorization endpoint — where user signs in
token_uri	Google's token endpoint — where code is exchanged for tokens
auth_provider_x509_cert_url	Google's public key URL (for JWT verification, not used here)
client_secret	Secret key from Google Cloud → GOOGLE_CLIENT_SECRET in .env
redirect_uris	Allowed redirect URIs registered in Google Cloud Console

Note: The `exchange_token_server.php` uses `redirect_uri = http://localhost:9090` while `get-google-token.php` uses `redirect_uri = http://localhost`. Both must be registered in the Google Cloud Console's "Authorized redirect URIs".

Overview

This system automatically sends a professional admin email with a PDF invoice every time a customer successfully places an order. It uses the **Gmail REST API** with **OAuth2 refresh token** — no SMTP password, no external mail library, no Events/Listeners.

Key Features

- ☒ Triggered directly from `OrderService::createOrder()` via `dispatch()`
 - ☒ Passes only `order_number` (string) to the job — no model serialization issues
 - ☒ Loads full order with all relations inside the job
 - ☒ Generates branded PDF invoice using `barryvdh/laravel-dompdf`
 - ☒ Sends HTML email + PDF attachment via Gmail REST API
 - ☒ Auto-retries 3 times with 30-second backoff on failure
 - ☒ Flags high-value orders (> 2000 DH) in subject and email body
 - ☒ Full logging at every step
-



Real File Structure

```
backend/
├── app/
│   ├── Jobs/
│   │   └── SendAdminOrderEmail.php    ← Queued job (entry point)
│   ├── Services/
│   │   ├── GmailService.php          ← OAuth2 token + Gmail REST API sender
│   │   ├── InvoiceService.php        ← PDF generation via DomPDF
│   │   └── OrderService.php          ← Dispatches job after order creation
│   └── Http/Controllers/
│       └── OrderController.php        ← Calls OrderService
├── resources/views/
│   └── invoices/
│       └── order.blade.php            ← Branded PDF template (Blade)
└── .env                              ← OAuth2 credentials + admin email
```

⚠️ **NO** Events/OrderCreated.php ⚠️ **NO** Listeners/NotifyAdminOrderReceived.php The job is dispatched directly from OrderService .

File 1 — GmailService.php

Path: app/Services/GmailService.php **Dependency:** guzzlehttp/guzzle (already in composer.json) **OAuth**

Scope: https://www.googleapis.com/auth/gmail.send

```

<?php

namespace App\Services;

use GuzzleHttp\Client;
use Illuminate\Support\Facades\Log;

class GmailService
{
    private Client $http;
    private string $clientId;
    private string $clientSecret;
    private string $refreshToken;
    private string $fromAddress;
    private string $fromName;

    public function __construct()
    {
        $this->http      = new Client(['timeout' => 30]);
        $this->clientId   = env('GOOGLE_CLIENT_ID', '');
        $this->clientSecret = env('GOOGLE_CLIENT_SECRET', '');
        $this->refreshToken = env('GOOGLE_REFRESH_TOKEN', '');
        $this->fromAddress = env('MAIL_FROM_ADDRESS', 'zakarialaalbad200@gmail.com');
        $this->fromName    = env('MAIL_FROM_NAME', 'Parfum Store');
    }

    // — Step 1: Exchange refresh token for a short-lived access token ———
    private function getAccessToken(): string
    {
        $response = $this->http->post('https://oauth2.googleapis.com/token', [
            'form_params' => [
                'client_id'     => $this->clientId,
                'client_secret' => $this->clientSecret,
                'refresh_token' => $this->refreshToken,
                'grant_type'    => 'refresh_token',
            ],
        ]);

        $data = json_decode($response->getBody()->getContents(), true);

        if (empty($data['access_token'])) {
            throw new \RuntimeException('Failed to obtain access token: ' . json_encode($data));
        }

        Log::info('[GmailService] Access token obtained.');
```

```

        return $data['access_token'];
    }

    // — Step 2: Build RFC 2822 MIME message (HTML + optional PDF) ———
    private function buildMimeMessage(
        string $to,
        string $subject,
```

```

        string $htmlBody,
        ?string $pdfContent = null,
        ?string $pdfFilename = null
    ): string {
        $from      = "{$this->fromName} <{$this->fromAddress}>";
        $boundary  = '==Boundary_' . md5(uniqid((string) rand(), true));
        $subject64 = '=?UTF-8?B?' . base64_encode($subject) . '?=';

        $mime = "From: {$from}\r\n";
        $mime .= "To: {$to}\r\n";
        $mime .= "Subject: {$subject64}\r\n";
        $mime .= "MIME-Version: 1.0\r\n";

        if ($pdfContent && $pdfFilename) {
            // multipart/mixed = HTML body + PDF attachment together
            $mime .= "Content-Type: multipart/mixed; boundary=\"{$boundary}\"\\r\\n\\r\\n";

            $mime .= "--{$boundary}\\r\n";
            $mime .= "Content-Type: text/html; charset=UTF-8\\r\\n";
            $mime .= "Content-Transfer-Encoding: base64\\r\\n\\r\\n";
            $mime .= chunk_split(base64_encode($htmlBody)) . "\\r\\n";

            $mime .= "--{$boundary}\\r\n";
            $mime .= "Content-Type: application/pdf; name=\"{$pdfFilename}\"\\r\\n";
            $mime .= "Content-Disposition: attachment; filename=\"{$pdfFilename}\"\\r\\n";
            $mime .= "Content-Transfer-Encoding: base64\\r\\n\\r\\n";
            $mime .= chunk_split(base64_encode($pdfContent)) . "\\r\\n";

            $mime .= "--{$boundary}--";
        } else {
            $mime .= "Content-Type: text/html; charset=UTF-8\\r\\n";
            $mime .= "Content-Transfer-Encoding: base64\\r\\n\\r\\n";
            $mime .= chunk_split(base64_encode($htmlBody));
        }

        // Gmail API requires URL-safe base64 (no + / or = padding)
        return rtrim(strtr(base64_encode($mime), '+/', '-_'), '=');
    }

// — Step 3: POST to Gmail REST API —————
public function sendEmail(
    string $to,
    string $subject,
    string $htmlBody,
    ?string $pdfContent = null,
    ?string $pdfFilename = null
): void {
    $accessToken = $this->getAccessToken();
    $rawMessage = $this->buildMimeMessage($to, $subject, $htmlBody, $pdfContent, $pdfFilename);

    $response = $this->http->post(
        'https://www.googleapis.com/gmail/v1/users/me/messages/send',
        [

```

```

        'headers' => [
            'Authorization' => "Bearer {$accessToken}",
            'Content-Type' => 'application/json',
        ],
        'json' => ['raw' => $rawMessage],
    ]
);

$result = json_decode($response->getBody()->getContents(), true);
Log::info("[GmailService] Email sent to {$to} – messageId: " . ($result['id'] ?? 'unknown'))
}
}

```

Method Summary

Method	Purpose
<code>getAccessToken()</code>	POSTs to <code>oauth2.googleapis.com/token</code> with refresh token → returns short-lived access token
<code>buildMimeMessage()</code>	Builds RFC 2822 MIME structure with HTML + PDF parts, encodes as URL-safe base64
<code>sendEmail()</code>	Orchestrates token fetch → builds MIME → POSTs to Gmail API

File 2 — SendAdminOrderEmail.php

Path: `app/Jobs/SendAdminOrderEmail.php`

```

<?php

namespace App\Jobs;

use App\Models\Order;
use App\Services\GmailService;
use App\Services\InvoiceService;
use Illuminate\Bus\Queueable;
use Illuminate\Contracts\Queue\ShouldQueue;
use Illuminate\Foundation\Bus\Dispatchable;
use Illuminate\Queue\InteractsWithQueue;
use Illuminate\Queue\SerializesModels;
use Illuminate\Support\Facades\Log;

class SendAdminOrderEmail implements ShouldQueue
{
    use Dispatchable, InteractsWithQueue, Queueable, SerializesModels;

    public int $tries = 3; // Auto-retry 3 times on failure
    public int $backoff = 30; // Wait 30s between retries

    // ✅ Accepts order_number (string) – NOT the Order model
    // This avoids serialization issues if model changes between dispatch and execution
    public function __construct(private readonly string $orderNumber) {}

    public function handle(GmailService $gmailService, InvoiceService $invoiceService): void
    {
        Log::info("SendAdminOrderEmail: Processing order #{$this->orderNumber}");

        // Load fresh order with all relations needed for email + PDF
        $order = Order::with(['items.product.images', 'shippingMethod', 'coupon'])
            ->where('order_number', $this->orderNumber)
            ->first();

        if (! $order) {
            Log::warning("SendAdminOrderEmail: Order {$this->orderNumber} not found.");
            return;
        }

        $adminEmail = env('MAIL_ADMIN_EMAIL');

        // Step 1: Generate PDF (graceful fallback – send email even if PDF fails)
        $pdfContent = $pdfFilename = null;
        try {
            $pdfContent = $invoiceService->generatePdf($order);
            $pdfFilename = "invoice-{$order->order_number}.pdf";
        } catch (\Throwable $e) {
            Log::warning("PDF generation failed: {$e->getMessage()} – continuing without attachment.");
        }

        // Step 2: Build HTML email body
        $isHighValue = (float) $order->total > 2000;
    }
}

```

```

$htmlBody    = $this->buildEmailHtml($order, $isHighValue);

// Step 3: Compose subject (flags high-value orders)
$urgentTag = $isHighValue ? '🔥 [HIGH VALUE] ' : '';
$subject    = "{$urgentTag}📄 New Order Received - #{ $order->order_number}";

// Step 4: Send via Gmail API
$gmailService->sendEmail(
    to:         $adminEmail,
    subject:     $subject,
    htmlBody:    $htmlBody,
    pdfContent:  $pdfContent,
    pdfFilename: $pdfFilename,
);

Log::info("SendAdminOrderEmail: Email sent to {$adminEmail} for order #{ $order->order_number }");

private function buildEmailHtml(Order $order, bool $isHighValue): string
{
    // Builds a full styled HTML invoice email inline
    // Includes: customer info, order items table, subtotal/discount/shipping/total
    // ...see actual file for full implementation
}

```

Key Design Decisions

Decision	Reason
Accepts string \$orderNumber not Order \$order	Avoids model serialization issues in queue
Loads order fresh inside handle()	Gets latest data at time of sending, not at dispatch time
\$tries = 3 + \$backoff = 30	Auto-retries on Gmail API timeout or network errors
PDF failure is non-fatal	Email still sends even if DomPDF fails

File 3 — InvoiceService.php

Path: `app/Services/InvoiceService.php` **Dependency:** `barryvdh/laravel-dompdf`


```
<?php

namespace App\Services;

use App\Models\Order;
use Barryvdh\DomPDF\Facade\Pdf;

class InvoiceService
{
    public function generatePdf(Order $order): string
    {
        $order->loadMissing(['items.product', 'shippingMethod']);

        $pdf = Pdf::loadView('invoices.order', ['order' => $order])
            ->setPaper('a4', 'portrait');

        return $pdf->output(); // Returns raw PDF binary
    }
}
```

- Loads the Blade view `resources/views/invoices/order.blade.php`
- Returns the raw binary PDF string (passed directly to `GmailService`)

File 4 — OrderService.php

Path: `app/Services/OrderService.php`

The dispatch happens at the very end of `createOrder()`, **after** the database transaction commits:

```
// Inside OrderService::createOrder() – at the end, after DB::transaction()

SendAdminOrderEmail::dispatch($order->order_number);
//           ↑
//       Passes order_number string only (not the model)
//       Job is queued in the database (jobs table)
//       Returns immediately – doesn't block the HTTP response

return $order;
```

Important: `dispatch()` is called **outside** the `DB::transaction()` block. This ensures:

- The order is fully committed before the job runs
 - The job won't process a half-created order
-

File 5 — invoices/order.blade.php

Path: `resources/views/invoices/order.blade.php`

A professional A4 branded PDF invoice using the brand color palette:

Style Element	Value
Brand color	#da2966 (pink/rose)
Dark color	#423835 (dark brown)
Font	DejaVu Sans (DomPDF compatible)
Paper	A4 portrait

Sections:

- Header: Brand name + invoice number + date
- Bill To: Customer name, email, phone, address
- Status badge: `pending` / `confirmed` / `shipped` / `delivered`
- Items table: Product name, variant (size), qty, unit price, subtotal
- Totals: Subtotal → Discount (if coupon) → Shipping → **Total**
- Footer: Thank you message + store contact

Environment Configuration (.env)

All variables used by the Gmail order notification system, with detailed explanation of each line:

```
# =====
# SECTION 1 – Laravel SMTP (fallback mailer, NOT used for order emails)
# =====

# Mail driver – set to smtp for Laravel's default Mail facade
MAIL_MAILER=smtp

# Gmail SMTP server hostname
MAIL_HOST=smtp.gmail.com

# SMTP port 587 = STARTTLS (recommended), 465 = SSL
MAIL_PORT=587

# Gmail account used for SMTP login authentication
MAIL_USERNAME=zakarialaalbad200@gmail.com

# Gmail App Password (16 characters, generated at myaccount.google.com/apppasswords)
# NOT the regular account password – a dedicated app-specific password
MAIL_PASSWORD=iottdrjftctaxbqr

# TLS encryption required for port 587
MAIL_ENCRYPTION=tls

# Default From: address used by Laravel's Mail facade (not used by GmailService)
MAIL_FROM_ADDRESS=zakarialaalbad200@gmail.com

# Display name shown in email clients for the sender
MAIL_FROM_NAME="Parfum Store"

# Admin email address for order notifications – read by SendAdminOrderEmail job
# This is the To: recipient of every admin order email
MAIL_ADMIN_EMAIL=zakarialaabad2005@gmail.com

# =====
# SECTION 2 – Google Gmail REST API (ACTIVE – used for order notifications)
# =====

# OAuth2 Client ID
# Source: Google Cloud Console → APIs & Services → Credentials → OAuth 2.0 Client IDs
# Used by GmailService::__construct() to identify the application to Google
GOOGLE_CLIENT_ID=379053566763-8ikbpbcb0mohpls89fddv859n992l9ghq.apps.googleusercontent.com

# OAuth2 Client Secret
# Source: Same Google Cloud Console credential – keep private, never commit to git
# Used by GmailService::getAccessToken() to authenticate the token exchange request
GOOGLE_CLIENT_SECRET=GOCS PX-61F1QJvsrPqXQtFid6vIMJYjKWIB

# OAuth2 Refresh Token
# Obtained ONCE via the OAuth authorization flow (user grants permission)
# Never expires unless manually revoked in Google account settings
# Used by GmailService::getAccessToken() – exchanged for a short-lived access token
# The access token (valid 1 hour) is then used to authenticate Gmail API calls
```

```

GOOGLE_REFRESH_TOKEN=1//03rLlukLSoSqCgYIARAAGAMSNwF-L9Ir...

# Full RFC 2822 From: header for the MIME message
# Format: "Display Name <gmail-address>" – MUST be the Gmail account that owns the OAuth token
# Used by GmailService::buildMimeMessage() to set the From: header in the raw email
GMAIL_FROM="Parfum Store <zakarialaalbad200@gmail.com>"

# Admin inbox that RECEIVES all order notification emails
# Used by SendAdminOrderEmail::handle() as the To: address passed to GmailService
# Different from the sender (zakarialaalbad200) – this is where emails are delivered
ADMIN_EMAIL=zakarialaabad2005@gmail.com

# =====
# SECTION 3 – Queue (required for async job processing)
# =====

# Queue driver – 'database' stores jobs in the 'jobs' table in MySQL
# Allows email to be sent in the background without blocking the checkout response
QUEUE_CONNECTION=database

```

Variable → Code Mapping (Exact Usage)

.env Variable	Read In File	Method	Role
GOOGLE_CLIENT_ID	GmailService.php	__construct()	Sent to Google to identify the OAuth2 app
GOOGLE_CLIENT_SECRET	GmailService.php	getAccessToken()	Authenticates token exchange POST request
GOOGLE_REFRESH_TOKEN	GmailService.php	getAccessToken()	POSTed to oauth2.googleapis.com/token → returns access_token
GMAIL_FROM	GmailService.php	buildMimeMessage()	Sets the From: header in the raw MIME email
ADMIN_EMAIL	SendAdminOrderEmail.php	handle()	Passed as \$to into GmailService::sendEmail()
MAIL_ADMIN_EMAIL	SendAdminOrderEmail.php	handle()	Fallback if ADMIN_EMAIL is not set
QUEUE_CONNECTION	Laravel framework	Queue dispatcher	Determines where dispatched jobs are stored

OAuth2 Token Flow in Detail

```
GmailService::getAccessToken()
|
| POST https://oauth2.googleapis.com/token
| Body:
|   grant_type    = refresh_token
|   client_id     = GOOGLE_CLIENT_ID      ← from .env
|   client_secret = GOOGLE_CLIENT_SECRET  ← from .env
|   refresh_token = GOOGLE_REFRESH_TOKEN  ← from .env
|
| ▼
Response: { "access_token": "ya29.xxx", "expires_in": 3599 }
|
| ▼ Used immediately in:
GmailService::sendEmail()
| POST https://www.googleapis.com/gmail/v1/users/me/messages/send
| Header: Authorization: Bearer ya29.xxx
| Body: { "raw": "<base64url MIME message>" }
|
| ▼
✅ Email delivered to ADMIN_EMAIL
```

Operation Flow

```
1. POST /api/v1/orders
   |
2. OrderController::store()
   |
3. OrderService::createOrder()
   |─ Validates items, stock, coupon, shipping
   |─ DB::transaction() → creates Order + OrderItems + decrements stock
   |─ SendAdminOrderEmail::dispatch($order->order_number)
   |
4. Job saved to `jobs` table in database
   Response returned to customer: ⚡ ~200ms
   |
5. Queue worker picks up job:
   php artisan queue:work
   |
6. SendAdminOrderEmail::handle()
   |─ Loads: Order::with(['items.product.images', 'shippingMethod', 'coupon'])
   |─ InvoiceService::generatePdf($order)
   |   └─ Pdf::loadView('invoices.order') → A4 PDF binary
   |─ buildEmailHtml($order) → styled HTML
   |─ GmailService::sendEmail(MAIL_ADMIN_EMAIL, subject, html, pdf)
   |
7. GmailService::getAccessToken()
   |─ POST https://oauth2.googleapis.com/token → access_token
   |
8. GmailService::buildMimeMessage()
   |─ RFC 2822 multipart/mixed MIME → URL-safe base64
   |
9. POST https://www.googleapis.com/gmail/v1/users/me/messages/send
   |
10. ✅ Admin receives email at MAIL_ADMIN_EMAIL
    Subject: 📧 New Order Received - #ORD-XXXXX
    Body: Customer info + order summary + items table
    Attachment: invoice-ORD-XXXXX.pdf
```

Queue Worker

The queue worker **must be running** for jobs to process:

```
cd backend
php artisan queue:work
```

Expected output per order:

```
2026-03-31 19:05:13 App\Jobs\SendAdminOrderEmail ..... RUNNING
2026-03-31 19:05:19 App\Jobs\SendAdminOrderEmail ..... 5s DONE
```

- **RUNNING** → job started processing
- **DONE** → email sent successfully
- **FAIL** → check logs, will auto-retry up to 3 times

For Production (keep worker alive with Supervisor):

```
[program:parfum-worker]
command=php /path/to/backend/artisan queue:work --sleep=3 --tries=3
autostart=true
autorestart=true
```

Troubleshooting

Job shows **FAIL** → check logs

```
Get-Content storage\logs\laravel.log | Select-Object -Last 50
```

Common Errors

Error	Cause	Fix
Failed to obtain access token	GOOGLE_REFRESH_TOKEN expired or missing	Re-run <code>php artisan google:get-token</code>
Order XXXX not found	Job ran before transaction committed	Already fixed — dispatch is outside transaction
MAIL_ADMIN_EMAIL not configured	Missing <code>.env</code> variable	Add <code>MAIL_ADMIN_EMAIL=admin@example.com</code>
PDF generation failed	DomPDF issue	Email still sends without attachment
Job stuck in RUNNING	Queue worker crashed	Restart <code>php artisan queue:work</code>

Verify System Works

```
# Check failed jobs
php artisan queue:failed

# Retry all failed jobs
php artisan queue:retry all

# Check specific env values are loaded
php artisan tinker --execute="echo env('MAIL_ADMIN_EMAIL');"
php artisan tinker --execute="echo env('GOOGLE_REFRESH_TOKEN') ? 'token set' : 'MISSING';"
```

✓ Verification Checklist

- ☐ `GOOGLE_REFRESH_TOKEN` is set in `.env` (not empty)
- ☐ `MAIL_ADMIN_EMAIL` is set to valid email in `.env`
- ☐ `QUEUE_CONNECTION=database` in `.env`
- ☐ `jobs` table exists (`php artisan migrate`)
- ☐ `php artisan queue:work` is running in terminal
- ☐ Order is created → queue shows `DONE` (not `FAIL`)
- ☐ Admin inbox receives email with PDF attachment